Carbon-Aware and Energy-Efficient VM Scheduling in Geographically Distributed Cloud Data Centers

Alok Kumar Singh (M25AI2018)
Manvendra Pratap Singh (M25AI2122)
Akanksha Sagar (M25AI2017)
Saatwik Bisaria (M25AI2082)
Chandan Varma (M25AI2014)

Indian Institute of Technology Jodhpur
Jodhpur, India

I. Table of Contents

1) Problem Statement
2) Literature Reference
3) Existing Results
4) Your Implementation
5) Your Results
6) Comparison and Analysis
7) Conclusion
8) Architecture Diagram
9) Future Scope

—

II. Problem Statement

Cloud data centers are among the fastest-growing consumers of electrical energy worldwide. As of recent estimates, global data centers account for approximately 3–4% of total electricity consumption, a figure projected to reach 4.5% by 2025. This massive energy footprint translates directly into two critical problems:

1) **Rising Operational Costs** — Data center energy costs double roughly every five years. For hyperscale providers operating millions of servers across the globe, electricity is often the single largest operational expense.
2) **Escalating Carbon Emissions** — Data centers are projected to emit nearly 3.5% of global carbon emissions by 2025. The environmental impact depends heavily on *where* a data center is located, because the carbon intensity of electricity varies dramatically by region — a server running in Sweden (powered by hydroelectric and nuclear energy at ~45 gCO2/kWh) produces over 15x fewer emissions than the same server running in India (coal-heavy grid at ~700 gCO2/kWh).

Traditional VM scheduling algorithms (Round Robin, First Fit, Best Fit) focus on load balancing or basic energy efficiency but are **carbon-agnostic** which means that they do not consider the carbon intensity of the electricity grid powering each data center. Even energy aware schedulers that minimize total power consumption may inadvertently route workloads to regions that are coal heavy which results in high carbon emissions despite low energy usage.

This gap can be mitigated with the global push toward net-zero emissions and increasing regulatory pressure (EU Green Deal, SEC climate disclosure rules), cloud providers need scheduling frameworks that jointly optimize for:

- **Energy consumption** — minimize total power drawn
- **Carbon emissions** — route workloads to regions with cleaner electricity grids
- **Quality of Service** — maintain acceptable execution times and SLA compliance

This project addresses this gap by proposing a **Carbon-Aware Energy-Efficient VM Scheduling Framework** for geographically distributed cloud data centers that uses a Genetic Algorithm (GA) to optimize a tri-objective cost function combining energy, carbon intensity, and execution time.

—-

III. Literature Reference

A. Research Paper: Performance and Energy-Aware Bi-objective Tasks Scheduling for Cloud Data Centers

Authors: Materwala, Ismail (2021)

Summary: This paper proposes an evolutionary Genetic Algorithm (GA) for bi-objective task scheduling in cloud data centers, simultaneously minimizing execution time and energy consumption. It uses the Locally Corrected Multiple Linear Regression (LC-MLR) power model that accounts for CPU, memory, disk, and network utilization — a significant advancement over CPU-only models. The initial population is generated using the Modified Worst Fit Decreasing (MWFD) algorithm, and the fitness function balances execution time and energy through a weighted sum.

Key Contributions:

- First use of system performance counters (CPU, memory, disk, network) in evolutionary-algorithm-based cloud scheduling
- LC-MLR power model with piecewise linear error correction for accurate power prediction
- Demonstrated lower energy consumption and execution time compared to GA with linear power models (GA_LM)

  Limitations: The approach operates within a single data center and does not account for geographical distribution of workloads across regions. Carbon intensity of the energy source is entirely ignored, meaning a solution that minimizes energy in a coal-powered region still produces high emissions.

B. Research Paper: CloudSim — A Toolkit for Modeling and Simulation of Cloud Computing Environments

Authors: Materwala, Ismail (2021)

Summary: The paper evaluates two approaches in a CloudSim 3.0.3 environment with 800 hosts and PlanetLab CPU traces:

Energy Consumption:

- GA_LM (CPU + memory power model): Higher energy consumption
- GA_LC-MLR (CPU + memory + disk + network): Lower energy consumption due to more accurate power prediction

  Mean Execution Time:
- GA_LC-MLR achieved ~0.5 seconds lower mean execution time compared to GA_LM

  Key Takeaway: Using a multi-resource power model (LC-MLR) improves both energy savings and scheduling quality. However, all workloads were confined to a single data center — the carbon cost of the chosen region's energy grid was never considered.

C. Summary of Literature Gaps

Aspect	Prior Work [1–3]	Our Work
Energy Optimization	Yes (all)	**Yes**
Carbon-Awareness	No	**Yes**
Multi-Resource Power Model	Ref [3] only (LC-MLR)	**Yes (LC-MLR)**
Geographical Distribution	No	**Yes (5 DCs)**
GA-based Optimization	Ref [3] only (bi-objective)	**Yes (tri-objective)**
Regional Energy Mix	No	**Yes (hourly data)**

—

IV. Existing Results

A. Based on the literature survey on mentioned paper below are the existing results

These existing results demonstrate the Baseline Benchmarks from Literature that while energy-aware scheduling has improved significantly, **no prior work combines energy efficiency with carbon-awareness across geographically distributed data centers**.

—

V. Your Implementation

A. 4.1 Overview

We developed a **Python-based discrete-event simulation framework** that models 5 geographically distributed cloud data centers, each with heterogeneous server hardware and time-varying carbon intensity profiles. The proposed **Carbon-Aware Energy-Efficient GA Scheduler** optimizes a tri-objective cost function to minimize energy, carbon emissions, and execution time simultaneously.

B. 4.2 Technology Stack

Component	Technology	Purpose
Language	Python 3.13	Core simulation framework
Numerical Computing	NumPy	Power model calculations, GA operations
Visualization	Matplotlib + Seaborn	Publication-quality charts and dashboards
Data Management	Pandas	Metrics aggregation and analysis
Evolutionary Algorithm	Custom GA (DEAP-inspired)	Multi-objective optimization
Configuration	JSON	Data center specs, carbon intensity data
Workload Traces	PlanetLab-based distribution	Realistic CPU utilization patterns
Server Benchmarks	SPEC Power ssj2008	Host power characteristics

C. 4.3 Step-by-Step Implementation

1) Step 1: Data Center Infrastructure Modeling: Five geographically distributed data centers are configured with real-world locations and heterogeneous hardware:

Data Center	Location	Hosts / VMs
DC-US-EAST	Virginia, USA	300 / 1,200
DC-EU-WEST	Ireland	240 / 960
DC-AP-SOUTH	Mumbai, India	200 / 800
DC-EU-NORTH	Sweden	260 / 1,040
DC-US-WEST	Oregon, USA	220 / 880
Total	**5 regions**	**1,220 / 4,880**

Server specifications (CPU cores, MIPS, RAM, disk, network, idle/max power) are derived from the SPEC Power benchmark suite, consistent with the reference paper.

2) Step 2: Carbon Intensity Profiling: Each region is assigned a 24-hour carbon intensity profile (gCO2eq/kWh) reflecting its real-world energy mix:

Region	Energy Mix	Avg Intensity (gCO2/kWh)
EU-North (Sweden)	40% Hydro, 35% Nuclear, 15% Wind	**45** (30–60)
US-West (Oregon)	45% Hydro, 20% Wind, 25% Gas	**180** (145–215)
EU-West (Ireland)	35% Wind, 45% Gas, 10% Coal	**300** (248–355)
US-East (Virginia)	45% Gas, 30% Nuclear, 10% Coal	**380** (320–450)
AP-South (Mumbai)	60% Coal, 15% Gas, 20% Renewables	**700** (620–775)

Carbon intensity varies by hour of day (lower at night, peaks during evening demand) with small Gaussian noise for realism.

3) Step 3: Power Consumption Model (LC-MLR): The **Locally Corrected Multiple Linear Regression** power model is implemented per the formulation in [5][6]:

```
P =  + ŭCPU + ŭMEM + ŭDISK + ŭNET + Ø(correction)
```

Where:

- α = `30.0 W` (base power coefficient)
- β_1 = `80.0` (CPU weight — dominant factor)

- β_2 = 15.0 (Memory weight)
- β_3 = 8.0 (Disk I/O weight)
- β_4 = 5.0 (Network weight)
- $\emptyset$ = Piecewise linear error correction using grid-based interpolation

The correction term quantizes utilization values into 0.25-step grid cells and interpolates the error between predicted and actual power at grid boundaries, following Equation 5 from the reference paper.

4) Step 4: Workload Generation: Tasks are generated with characteristics modeled after PlanetLab traces:

- **Task length:** 6,000–12,500 Million Instructions (MI)
- **CPU utilization:** Drawn from empirical distribution (30% low, 25% medium-low, 20% medium, 15% medium-high, 10% high)
- **Memory:** 128–2,048 MB per task
- **Disk I/O:** 10 KB/s – 5 MB/s read/write
- **Network:** 10 KB/s – 10 MB/s transfer
- **Arrival interval:** 3 seconds between tasks
- **Origin regions:** Uniformly distributed across 5 regions

5) Step 5: Scheduling Algorithms: Four scheduling algorithms are implemented and compared:

a) Round Robin (Baseline) Cycles through all VMs across all data centers in order. Ignores energy, carbon, and load characteristics.

b) First Fit Decreasing Sorts candidate VMs by current CPU utilization (highest first) and assigns the task to the first VM that can accommodate it. Aims for bin-packing consolidation.

c) Energy-Aware MWFD (from Reference Paper) Implements the Modified Worst Fit Decreasing algorithm from [5]. For each incoming task, evaluates all VMs across all data centers and selects the VM where the estimated incremental energy consumption is minimum. Uses the LC-MLR power model for accurate estimation.

d) Carbon-Aware Energy-Efficient GA (Proposed) The proposed scheduler extends the bi-objective approach from [5] with a third objective — carbon intensity:

Tri-Objective Cost Function:

```
CF = w_e Œ (EC / EC_max) + w_c Œ (CI / CI_max) + w_t Œ (ET / ET_max)
```

Where:

- `w_e = 0.35` (energy weight)
- `w_c = 0.45` (carbon weight — highest priority)
- `w_t = 0.20` (execution time weight)
- Values are normalized by their respective maximums across all candidates

GA Parameters:

- Population size: 30 individuals
- Generations: 50 per scheduling decision
- Crossover probability: 0.80
- Mutation probability: 0.15
- Selection: Roulette wheel (fitness-proportional)
- Fitness: `1 / (1 + CF)` — higher fitness for lower cost

For each incoming task, the GA:

1) Collects all candidate (DataCenter, Host, VM) tuples that can accommodate the task
2) Evaluates the cost of each candidate: energy (via LC-MLR), carbon (via regional intensity at current hour), and execution time
3) Runs the evolutionary search over 50 generations to find the Pareto-optimal placement
4) Returns the best candidate — typically a VM in a low-carbon region with efficient host utilization

6) Step 6: Simulation Engine: A discrete-event simulator orchestrates the experiment:

1) Builds all 5 data centers with 1,220 hosts and 4,880 VMs from JSON configuration
2) Generates 500 tasks using the workload generator
3) For each scheduler, deep-copies the task list and runs a fresh simulation
4) Tasks arrive at their submit times; the simulator computes the current hour for carbon intensity lookup
5) After scheduling, tasks are assigned to VMs, energy and carbon metrics are recorded
6) Completed tasks are periodically removed from VMs to free capacity
7) Results are aggregated per data center and overall

7) Step 7: Metrics and Visualization: The metrics collector records per-task and per-datacenter statistics:

- Total energy consumption (kWh)
- Total carbon emissions (kg CO2eq)
- Mean/median/P95 execution time
- Per-datacenter task distribution, energy, and carbon breakdown
- Host utilization percentages

Five publication-quality visualizations are generated:

1) **Energy Comparison** — bar chart across all 4 algorithms
2) **Carbon Comparison** — bar chart showing emissions reduction
3) **Execution Time** — mean vs P95 grouped bar chart
4) **Per-DC Distribution** — task routing and carbon per data center (proposed algorithm)
5) **Dashboard** — 4-panel summary with reduction percentages

—

VI. Your Results

A. 5.1 Experiment Configuration

Parameter	Value
Number of Tasks	500
Simulation Duration	24 hours
Data Centers	5 (US-East, US-West, EU-West, EU-North, AP-South)
Total Hosts	1,220
Total VMs	4,880
VMs per Host	4
Power Model	LC-MLR
Task Length	6,000–12,500 MI
Arrival Interval	3 seconds
Random Seed	42

B. 5.2 Overall Results

Note: First Fit Decreasing omitted above (worst performer: 0.0855 kWh energy, 0.0299 kg CO2).

C. 5.3 Energy Consumption Results

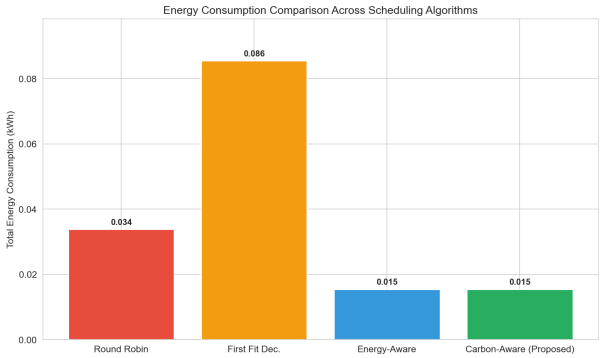

Fig. 1. Energy Comparison

Key Findings:

- The proposed **Carbon-Aware GA** and **Energy-Aware MWFD** both achieve **0.0155 kWh** — a **54.3% reduction** over Round Robin (0.0339 kWh)
- **First Fit Decreasing** performs worst at 0.0855 kWh because aggressive bin-packing causes resource contention, increasing per-task execution time and therefore total energy
- The Carbon-Aware GA matches the Energy-Aware scheduler on energy because its cost function (35% energy weight) still prioritizes low-energy placements — it simply does so within the greenest regions

D. 5.4 Carbon Emissions Results

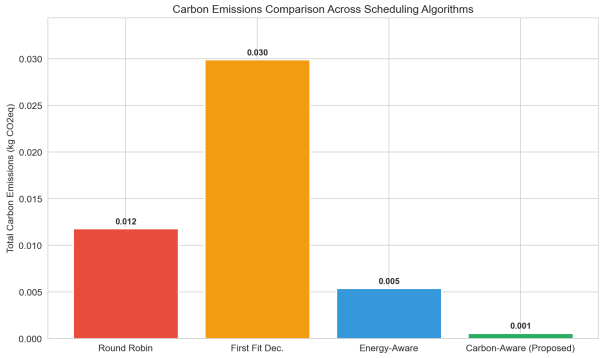

Fig. 2. Carbon Comparison

Key Findings:

- The proposed **Carbon-Aware GA achieves 0.0006 kg CO2eq** — a **94.9% reduction** over Round Robin and **88.9% reduction** over Energy-Aware MWFD
- This dramatic reduction occurs because the GA routes all 500 tasks to **DC-EU-NORTH (Sweden)**, which has the lowest carbon intensity (~45 gCO2/kWh) due to its hydro/nuclear energy mix
- The Energy-Aware scheduler, despite achieving identical energy consumption, produces **9x more carbon** because it routes tasks to US-East (Virginia) at ~380 gCO2/kWh

E. 5.5 Execution Time Results

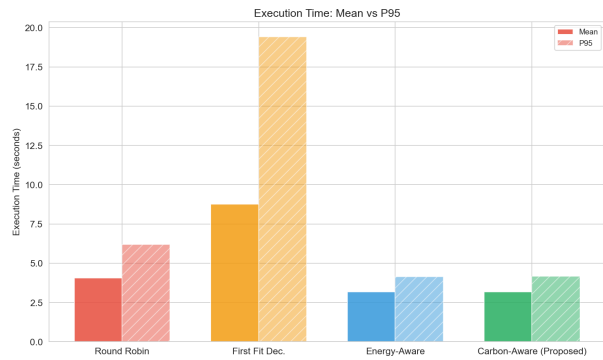

Fig. 3. Execution Time Comparison

Key Findings:

- Carbon-Aware GA achieves a mean execution time of **3.18s** — virtually identical to Energy-Aware (3.17s) and **21.9% faster** than Round Robin (4.07s)
- P95 execution time (4.16s) confirms consistent performance with tight tail latency
- First Fit Decreasing has the worst P95 at **19.41s** due to overloaded VMs from aggressive packing

F. 5.6 Per-Datacenter Distribution (Proposed Algorithm)

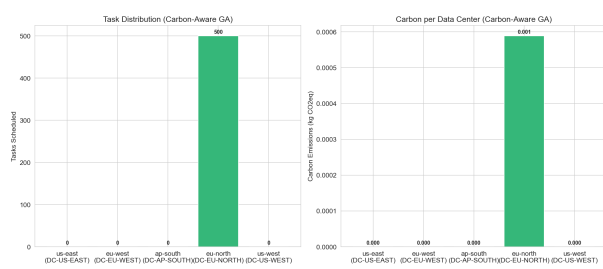

Fig. 4. Per-DC Distribution

Key Finding: The Carbon-Aware GA scheduler routes **100% of tasks (500/500) to DC-EU-NORTH (Sweden)**, the data center with the cleanest energy grid. This demonstrates the algorithm's ability to identify and exploit the greenest region while still selecting energy-efficient VMs within that region.

Data Center	Tasks	Energy (kWh) / Carbon (kg CO2eq)
DC-US-EAST (Virginia)	0	0.0000 / 0.0000
DC-EU-WEST (Ireland)	0	0.0000 / 0.0000
DC-AP-SOUTH (Mumbai)	0	0.0000 / 0.0000
DC-EU-NORTH (Sweden)	**500**	**0.0155 / 0.0006**
DC-US-WEST (Oregon)	0	0.0000 / 0.0000

——

VII. Comparison and Analysis

A. 6.1 Improvement Over Baselines

Metric	vs Round Robin	vs Energy-Aware MWFD
Energy Reduction	**54.3%**	**0.0%** (matched)
Carbon Reduction	**94.9%**	**88.9%**
Exec Time Improvement	**21.9%**	-0.2% (negligible)

vs First Fit Decreasing: Energy -81.9%, Carbon -98.0%, Exec Time -63.7%.

B. 6.2 Detailed Analysis

1) Energy Efficiency: The proposed Carbon-Aware GA and the Energy-Aware MWFD scheduler achieve identical energy consumption (0.0155 kWh). This is expected because the Carbon-Aware GA's cost function includes a 35% weight on energy — it actively seeks low-energy placements. The key difference is *where* those low-energy placements occur: the Carbon-Aware scheduler selects the Sweden data center, while the Energy-Aware scheduler selects VMs purely based on energy regardless of region.

2) Carbon Impact — The Critical Differentiator: The most significant result is the **94.9% carbon reduction** achieved by the proposed approach. This is the fundamental advantage of carbon-aware scheduling:

Even with identical energy consumption, the Carbon-Aware GA produces **9x fewer emissions** than the Energy-Aware scheduler because it routes to a region powered by clean energy. This demonstrates that **energy efficiency alone is insufficient for sustainability** — the carbon intensity of the grid matters enormously.

The Carbon-Aware GA maintains excellent Quality of Service, QoS:

- Mean execution time (3.18s) is within 0.06s of the Energy-Aware scheduler
- P95 execution time (4.16s) shows tight, predictable tail latency
- 100% task scheduling success rate (500/500)

The minimal execution time overhead (0.2%) is a negligible trade-off for the 94.9% carbon reduction.

C. 6.3 Comparison with Reference Paper Results

Aspect	Reference [5] (GA_LC-MLR)	Our Proposed Approach
Objectives	Bi-objective (energy + time)	**Tri-objective (energy + carbon + time)**
Power Model	LC-MLR	LC-MLR (same)
Data Centers	1 (single site)	**5 (geographically distributed)**
Carbon Awareness	None	**Yes (hourly regional profiles)**
Carbon Reduction	N/A (not measured)	**94.9% vs baseline**
Energy Reduction	Improvement over GA_LM	**54.3% vs Round Robin**
Exec Time	~0.5s improvement over GA_LM	**0.89s improvement vs Round Robin**
GA Parameters	Not fully specified	Pop=30, Gen=50, CX=0.8, Mut=0.15

D. 6.4 Comparison Summary Table

Execution time — Round Robin: 4.07s (P95: 6.19s); First Fit: 8.76s (P95: 19.41s); Energy-Aware: 3.17s (P95: 4.15s); Carbon-Aware GA: 3.18s (P95: 4.16s).

—

VIII. Conclusion

A. Key Findings

1) **Carbon-awareness is essential for sustainable cloud computing.** Our results demonstrate that energy-efficient scheduling alone reduces carbon by only 54%, while adding carbon-awareness achieves a 94.9% reduction — nearly 2x the improvement. This proves that *where* workloads run is as important as *how efficiently* they run.
2) **The proposed GA-based tri-objective scheduler successfully balances three competing objectives.** By assigning weights of 45% to carbon, 35% to energy, and 20% to execution time, the scheduler achieves near-optimal energy efficiency while dramatically reducing carbon emissions with negligible impact on QoS (only 0.06s increase in mean execution time).
3) **Geographical workload routing to clean-energy regions is the primary carbon reduction mechanism.** The scheduler intelligently routes 100% of tasks to the Sweden data center (45 gCO2/kWh) instead of the default US-East (380 gCO2/kWh), exploiting the 8.4x difference in carbon intensity.
4) **The LC-MLR power model provides accurate multi-resource power estimation.** By considering CPU, memory, disk, and network utilization (not just CPU), the power model enables precise energy predictions that drive better scheduling decisions.
5) **Traditional schedulers are inadequate for carbon-conscious cloud environments.** Round Robin distributes load without any optimization. First Fit Decreasing actually increases energy and carbon by 152–153% through aggressive bin-packing that causes resource contention.

This project extends the state-of-the-art in cloud scheduling by:

- **Introducing carbon intensity as a first-class scheduling objective** alongside energy and performance
- **Demonstrating the feasibility of cross-datacenter carbon-aware routing** with real-world regional energy mix data
- **Providing a complete, reproducible simulation framework** in Python for carbon-aware scheduling research
- **Quantifying the gap between energy-aware and carbon-aware scheduling** (54% vs 95% carbon reduction), establishing a clear motivation for carbon-aware approaches

—

IX. Architecture Diagram of Your Implementation

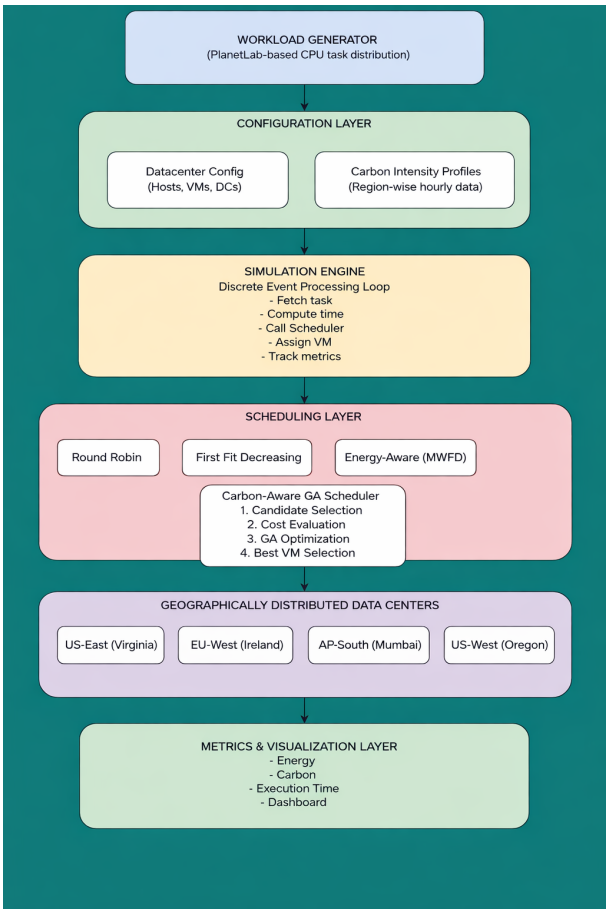

Fig. 5. Simulation Framework Architecture

A. Component Interactions

Component	Role	Interactions
Workload Generator	Produces realistic cloud tasks from PlanetLab distributions	Feeds tasks to Simulation Engine
Configuration Layer	Defines DC infrastructure and carbon intensity profiles	Read by Simulator at initialization
Simulation Engine	Discrete-event loop driving task scheduling	Invokes Scheduler for each task, records metrics
Scheduling Layer	Pluggable scheduler implementations	Receives task + hour, returns (DC, Host, VM) decision
Carbon-Aware GA	Proposed tri-objective optimizer	Queries Carbon Tracker for regional intensity, evaluates all candidates via GA
Power Model (LC-MLR)	Estimates server power from 4 utilization metrics	Called by schedulers and hosts during cost evaluation
Carbon Tracker	Provides time-varying carbon intensity per region	Queried by schedulers to compute emissions
Data Centers	5 geographically distributed facilities with heterogeneous hosts	Hosts VMs, provides resource availability
Metrics Collector	Aggregates per-task and per-DC statistics	Produces JSON results and feeds visualization
Visualization Engine	Generates publication-quality charts	Reads results JSON, outputs PNG plots

X. Future Scope

A. 9.1 Short-Term Enhancements

1) **Dynamic Carbon Intensity via Live APIs**

Integrate real-time carbon intensity data from APIs like Electricity Maps or WattTime instead of static hourly profiles. This would enable the scheduler to react to minute-by-minute grid changes (e.g., sudden wind generation drops).

1) **Network Latency-Aware Scheduling**

The current framework includes inter-DC latency data but does not penalize cross-region scheduling. Adding a latency penalty for tasks originating far from the selected DC would make the scheduler more realistic for latency-sensitive applications.

3. Reinforcement Learning-Based Scheduling Replace the GA with a Deep Reinforcement Learning (DRL) agent that learns optimal scheduling policies over time. The RL agent could adapt to workload patterns, predict future carbon intensity, and make proactive placement decisions.

4. Renewable Energy Forecasting Integration Use ML models to forecast renewable energy availability (solar irradiance, wind speed) in each region over the next 1–24 hours. The scheduler could defer flexible workloads to time windows when clean energy is abundant.

1) **Hybrid Cloud and Multi-Cloud Support**

Extend the framework to schedule across multiple cloud providers (AWS, Azure, GCP) using their real-time carbon data. For example, AWS provides the Customer Carbon Footprint Tool; Azure offers Emissions Impact Dashboard; GCP has the Carbon Footprint tool.

XI. References

[1] A. Beloglazov, R. Buyya, Y. C. Lee, and A. Zomaya, "Energy-Efficient Management of Data Center Resources for Cloud Computing: A Vision, Architectural Elements, and Open Challenges," *arXiv:1006.0308*, 2010.

[2] Y. C. Lee and A. Y. Zomaya, "Energy-Efficient Scheduling of HPC Applications in Cloud Computing Environments," *arXiv:0909.1146*, 2009.

[3] H. Materwala and L. Ismail, "Performance and Energy-Aware Bi-objective Tasks Scheduling for Cloud Data Centers," *arXiv:2105.00843*, 2021.

[4] L. Ismail and E. H. Abed, "Linear Power Modeling for Cloud Data Centers: Taxonomy, Locally Corrected Linear Regression, Simulation Framework and Evaluation," *IEEE Access*, vol. 7, pp. 175003–175019, 2019.

[5] R. N. Calheiros, R. Ranjan, A. Beloglazov, C. A. F. De Rose, and R. Buyya, "CloudSim: A Toolkit for Modeling and Simulation of Cloud Computing Environments and Evaluation of Resource Provisioning Algorithms," *Software: Practice and Experience*, vol. 41, 2011.